

BotStrike

Sistema de Trading Algoritmico para Strike Finance Perpetuals

Estrategias: Mean Reversion | Trend Following | Market Making

Microestructura: VPIN | Hawkes Process | Avellaneda-Stoikov

Kyle Lambda: Market Impact Estimation + Adverse Selection

Activos: BTC-USD | ETH-USD | ADA-USD

Arquitectura modular async/await con 12+ modulos independientes

Task Supervisor con auto-restart de tasks no-criticos

Backtesting realista tick-by-tick con datos reales de Strike

Paper trading con pipeline identico al live

Dashboard Streamlit en tiempo real

Documentacion Tecnica Completa

Marzo 2026

Indice

1. Vision General del Sistema	3
1.1 Objetivo	3
1.2 Caracteristicas Principales	3
1.3 Activos Soportados	3
2. Arquitectura del Sistema	5
2.1 Modulos del Sistema	5
2.2 Flujo de Datos en Tiempo Real	5
2.3 Modos de Operacion	6
3. Deteccion de Regimen de Mercado	7
3.1 Regimenes Definidos	7
3.2 Senales de Clasificacion	7
3.3 Thresholds Adaptativos	7
3.4 Suavizado Anti-Whipsaw	7
4. Estrategia: Mean Reversion	8
4.1 Logica de Entrada	8
4.2 Threshold Dinamico	8
4.3 Stop Loss y Take Profit	8
4.4 Signal Strength	8
4.5 Senal de Salida	8
4.6 Order Book Imbalance (Confirmacion)	9
4.7 Kelly Criterion (Sizing Optimo)	9
4.8 Proteccion de Microestructura	9
5. Estrategia: Trend Following	10
5.1 Senales de Entrada	10
5.2 Trailing Stop	10
5.3 Take Profit	10
5.4 Ajuste por Fuerza de Tendencia	10
6. Estrategia: Market Making	11
6.1 Motor Avellaneda-Stoikov Mejorado	11
6.2 Ajuste Dinamico de Gamma	11
6.3 Filtros de Seguridad	11
6.4 Unwind de Inventario	11

7. Indicadores de Microestructura	13
7.1 VPIN (Volume-Synchronized Probability of Informed Trading)	13
7.2 Hawkes Process (Deteccion de Picos de Actividad)	13
7.3 Kyle Lambda (Market Impact Estimation)	13
7.4 Risk Score Combinado	14
8. Gestion de Riesgo	15
8.1 Capas de Proteccion	15
8.2 Position Sizing	15
8.3 Reduccion por Perdidas Consecutivas	15
9. Gestion de Portfolio	16
9.1 Asignacion por Regimen	16
9.2 Factores de Ajuste	16
9.3 Desactivacion de Estrategias	16
10. Ejecucion de Ordenes y Paper Trading	17
10.1 Smart Order Router	17
10.2 Fill Probability Model	17
10.3 Modelo de Slippage Avanzado (8 Componentes)	17
10.4 Paper Trading Simulator	17
11. Sistema de Backtesting	19
11.1 Backtester Basico (Bar-by-Bar)	19
11.2 Backtester Realista (Tick-by-Tick)	19
11.3 Metricas de Resultado	19
12. Integracion con Strike Finance	20
12.1 Autenticacion Ed25519	20
12.2 WebSocket Dual	20
12.3 Rate Limiting	20
13. Pipeline de Datos	21
13.1 Recoleccion Continua	21
13.2 Formato de Almacenamiento	21
14. Configuracion y Parametros	22
14.1 Parametros Globales (TradingConfig)	22
14.2 Parametros por Simbolo (SymbolConfig)	22
15. Tick Quality Guards (WebSocket)	23
15.1 Guards Implementados	23
15.2 Diseno	23

16. Microprice y Order Book Alpha	24
16.1 Microprice (Stoikov 2018)	24
16.2 Order Book Imbalance (OBI)	24
17. Modelos Cuantitativos Avanzados	25
17.1 Volatility Targeting	25
17.2 Kelly Criterion (Half-Kelly Capped)	25
17.3 Risk of Ruin	25
17.4 Monte Carlo Bootstrap	25
17.5 Correlation Regime	25
17.6 Risk Parity (Covarianza)	26
18. Execution Intelligence	27
18.1 Queue Position Model	27
18.2 Spread Predictor	27
18.3 Trade Intensity Model (Bidireccional)	27
18.4 TWAP Engine	27
18.5 Execution Analytics	27
18.6 Slippage Real Measurement	27
19. Testing y Calidad deCodigo	29
19.1 Suites de Tests	29
19.2 Auditorias de Codigo Realizadas	29
19.3 Revision Cuantitativa	29

1. Vision General del Sistema

BotStrike es un sistema de trading algoritmico de grado profesional disenado para operar perpetual futures en Strike Finance, un exchange descentralizado de derivados. El sistema implementa tres estrategias complementarias que operan simultaneamente, adaptandose dinamicamente a las condiciones del mercado mediante deteccion automatica de regimen.

1.4 Objetivo

Generar retornos consistentes en mercados de criptomonedas mediante la combinacion de estrategias no correlacionadas (mean reversion, trend following, market making) que se complementan en distintos regimenes de mercado. El sistema prioriza la gestion de riesgo y la proteccion de capital sobre la maximizacion de retornos.

1.5 Caracteristicas Principales

- 3 estrategias simultaneas con asignacion dinamica de capital por regimen
- Microestructura avanzada: VPIN, Hawkes Process, Avellaneda-Stoikov mejorado
- Microprice (Stoikov 2018): fair value superior al mid-price en todas las decisiones
- Order Book Imbalance: alpha multi-nivel con exponential decay y delta tracking
- Execution Intelligence: Smart Router, Fill Probability, Queue Model, TWAP, Spread Predictor
- Modelos cuantitativos: Volatility Targeting, Kelly Criterion, Risk of Ruin, Monte Carlo
- Risk Parity: covarianza rolling + inverse-vol weighting para allocation optima
- Correlation Regime: detecta stress ($\text{corr} > 0.85$) y reduce exposicion automaticamente
- Gestion de riesgo multicapa: drawdown, exposure, circuit breaker, funding rate
- Paper trading con pipeline identico al live (sin divergencia de comportamiento)
- Backtesting realista tick-by-tick con datos reales de Strike Finance
- Dashboard Streamlit en tiempo real con 4 paginas de monitoreo
- Recoleccion continua de datos de mercado (WebSocket + REST) a Parquet
- Base de datos SQLite de trades con analytics multi-dimensional
- Walk-forward optimization y stress testing
- Slippage real measurement: tracking de expected vs fill price en cada trade
- Kyle Lambda: estimacion de impacto permanente de mercado ($\text{Cov}(dP, Q) / \text{Var}(Q)$)
- Adverse Selection Measurement: mark-to-market de fills tras T+5min
- Impact Stress: bloqueo/reduccion automatica cuando lambda es extremo
- Task Supervisor: auto-restart de tasks no-criticos, shutdown tras 3 crashes criticos
- Slippage avanzado de 8 componentes (incluye permanent impact via Kyle Lambda)

1.6 Activos Soportados

Simbolo	Leverage	Posicion Max	VPIN Bucket	Estrategia Primaria
---------	----------	--------------	-------------	---------------------

BTC-USD	10x	\$20,000	\$50,000	Todas
ETH-USD	15x	\$15,000	\$10,000	Todas
ADA-USD	20x	\$5,000	\$500	Todas

2. Arquitectura del Sistema

BotStrike sigue una arquitectura modular basada en eventos con 12+ modulos independientes. Cada modulo tiene una responsabilidad unica y se comunica con los demas a traves de interfaces bien definidas. El sistema es completamente asincrono (asyncio) para manejar multiples fuentes de datos y simbolos concurrentemente. Un Task Supervisor monitorea todos los loops criticos: si un task no-critico (metrics, data_refresh) crashea, se reinicia automaticamente. Si un task critico falla 3 veces, el sistema hace shutdown seguro.

2.4 Modulos del Sistema

Modulo	Directorio	Responsabilidad
Configuracion	config/	Parametros globales, por simbolo y por estrategia
Core Types	core/types.py	Enums, dataclasses compartidas (Signal, Order, Position)
Market Data	core/market_data.py	Recoleccion OHLCV, indicadores tecnicos, deteccion stale
Indicadores	core/indicators.py	ATR, EMA, RSI, ADX, Z-score, Bollinger, momentum
Regimen	core/regime_detector.py	Clasificacion: RANGING, TRENDING, BREAKOUT
Microestructura	core/microstructure.py	VPIN, Hawkes, A-S engine, Kyle Lambda
Microprice	core/microprice.py	Fair value L1, Multi-Level, Adjusted (intensity+OBI)
OBI Alpha	core/orderbook_alpha.py	Order Book Imbalance multi-nivel con delta
Quant Models	core/quant_models.py	VolTarget, Kelly, RoR, MonteCarlo, CorrRegime
Estrategias	strategies/	Mean Reversion, Trend Following, Market Making
Riesgo	risk/risk_manager.py	Drawdown, exposure, RoR, VolTarget, Kelly, CorrStress
Portfolio	portfolio/portfolio_manager.p	Regime allocation + Risk Parity blend
Smart Router	execution/smart_router.py	Fill prob, queue, TWAP, spread pred, analytics
Ejecucion	execution/	Order engine, paper simulator, slippage model
Exchange	exchange/	REST client Ed25519, WebSocket market+user
Backtesting	backtesting/	Backtester bar-by-bar y tick-by-tick realista
Trade DB	trade_database/	SQLite persistence, analytics adapter
Dashboard	dashboard/	Streamlit multi-page (live, backtest, risk, admin)

2.5 Flujo de Datos en Tiempo Real

El flujo principal del sistema en modo live/paper sigue este ciclo cada 5 segundos (MR/TF) o cada 500ms (MM):

- 1. WebSocket recibe trades y orderbook tick-by-tick de Strike Finance
- 2. MarketDataCollector agrega ticks en barras OHLCV de 1 minuto con indicadores
- 3. MicrostructureEngine actualiza VPIN y Hawkes por cada trade
- 3b. KyleLambdaEstimator computa impacto permanente via $Cov(dP, Q)/Var(Q)$ rolling
- 4. TradeIntensityModel clasifica trades como buy/sell y calcula intensidad bidireccional
- 5. RegimeDetector clasifica el mercado (RANGING / TRENDING / BREAKOUT)
- 6. MicropriceCalculator computa fair value ajustado (superior al mid_price)

- 7. OrderBookImbalance calcula presion de compra/venta multi-nivel
- 8. PortfolioManager calcula allocation con Risk Parity blend + regime weights
- 9. Cada estrategia genera senales con OBI confirmation + Kelly sizing
- 10. RiskManager valida: RoR, VolTarget, CorrStress, impact stress, drawdown, micro, funding
- 11. SmartOrderRouter decide limit vs market por modelo de costos
- 12. OrderExecutionEngine (o PaperSimulator) ejecuta con slippage tracking
- 13. ExecutionAnalytics mide implementation shortfall y calibra modelos

2.6 Modos de Operacion

Modo	Comando	Descripcion
Live Trading	python main.py	Ordenes reales al exchange Strike Finance
Paper Trading	python main.py --paper	Datos reales de mainnet, fills simulados, PnL completo
Dry Run	python main.py --dry-run	Monitoreo live sin ordenes ni simulacion
Backtest Basico	python main.py --backtest	Bar-by-bar con datos sinteticos o CSV
Backtest Realista	python main.py --backtest-realistic	Tick-by-tick con microestructura completa
Backtest Real	python main.py --backtest-real	Con datos recolectados de Strike
Stress Test	python main.py --backtest-stress	Inyeccion de eventos extremos
Walk-Forward	python main.py --walk-forward	Validacion out-of-sample por folds
Optimizer	python main.py --optimize	Grid search de parametros
Data Collector	python main.py --collect-data	Recoleccion continua a Parquet
Dashboard	python main.py --dashboard	Streamlit en localhost:8501

3. Deteccion de Regimen de Mercado

El RegimeDetector clasifica continuamente el estado del mercado para cada simbolo. Esta clasificacion determina que estrategias se activan y cuanto capital se les asigna. El detector usa multiples senales con thresholds adaptativos por activo y un mecanismo de suavizado para evitar whipsaws (cambios bruscos).

3.5 Regimenes Definidos

Regimen	Condicion	Estrategia Primaria	Peso MM
RANGING	Baja volatilidad, ADX bajo	Mean Reversion (45%)	45%
TRENDING_UP	ADX alto, momentum positivo, EMA c	Trend Following (70%)	20%
TRENDING_DOWN	ADX alto, momentum negativo, EMA c	Trend Following (70%)	20%
BREAKOUT	Alta volatilidad + momentum fuerte	Trend Following (85%)	10%
UNKNOWN	Datos insuficientes	Balanceado (30/40/30)	30%

3.6 Senales de Clasificacion

- Volatility Percentile (vol_pct): percentil de volatilidad en ventana de 100 barras
- ADX (Average Directional Index): fuerza de tendencia de 0 a 100
- Momentum 20 periodos: retorno porcentual en las ultimas 20 barras
- EMA Crossover: cruce de EMA rapida (12) sobre EMA lenta (26)

3.7 Thresholds Adaptativos

Los thresholds no son fijos - se recalculan continuamente basandose en los ultimos 500 datos de cada activo. Esto permite que BTC (menos volatil relativamente) y ADA (muy volatil) tengan umbrales apropiados a su naturaleza:

- vol_low: percentil 30 de la distribucion historica de vol_pct (min 0.2)
- vol_high: percentil 75 de la distribucion historica de vol_pct (max 0.9)
- adx_trend: percentil 60 de ADX historico (min 20.0)
- mom_threshold: percentil 65 de |momentum| historico (min 0.005)

3.8 Suavizado Anti-Whipsaw

Para evitar cambios de regimen erraticos que causarian entrada/salida constante de estrategias, el detector requiere 2 detecciones consecutivas del mismo regimen para confirmar un cambio. Si la deteccion actual no coincide con la anterior, mantiene el ultimo regimen estable.

4. Estrategia: Mean Reversion

La estrategia Mean Reversion opera bajo la premisa de que los precios tienden a regresar a su media estadística. Cuando el precio se aleja significativamente (medido por Z-score), la estrategia abre una posición esperando la reversion.

Regimen Optimo

RANGING - Mercados laterales donde el precio oscila alrededor de una media estable.

4.9 Logica de Entrada

Se generan senales de entrada cuando se cumplen TODAS las condiciones:

Direccion	Z-score	RSI	Datos Minimos
LONG (compra)	< -threshold (debajo de media)	< 35 (sobreventa)	lookback barras (100)
SHORT (venta)	> +threshold (encima de media)	> 65 (sobrecompra)	lookback barras (100)

4.10 Threshold Dinamico

El threshold de Z-score no es fijo. Se ajusta segun la volatilidad relativa actual (vol_pct):

```
entry_threshold = mr_zscore_entry * (0.8 + 0.4 * vol_pct)
```

Ejemplo con mr_zscore_entry = 2.0:

```
Baja volatilidad (vol_pct=0.1): threshold = 2.0 * 0.84 = 1.68 (mas senales)
Media (vol_pct=0.5):           threshold = 2.0 * 1.00 = 2.00 (normal)
Alta volatilidad (vol_pct=0.9): threshold = 2.0 * 1.16 = 2.32 (filtra ruido)
```

4.11 Stop Loss y Take Profit

Ambos son dinamicos basados en ATR (Average True Range):

- Stop Loss = precio +/- ATR x 1.5 (mr_atr_mult_sl)
- Take Profit = precio +/- ATR x 2.5 (mr_atr_mult_tp)
- Ratio riesgo/recompensa: 1:1.67

4.12 Signal Strength

La fuerza de la senal (0 a 1) es proporcional a cuanto se alejo el precio de la media. Esto permite que el sistema de ejecucion ajuste el tipo de orden:

```
strength = min(|zscore| / (entry_threshold * 2), 1.0)
```

4.13 Senal de Salida

Cuando el Z-score regresa hacia la media ($|zscore| < mr_zscore_exit = 0.5$), se genera una senal de cierre. Esto

captura la reversion sin esperar a que SL/TP se activen.

4.14 Order Book Imbalance (Confirmacion)

MR usa el OBI como filtro de confirmacion. Si la imbalance del orderbook favorece la direccion del trade (buy pressure para longs, sell pressure para shorts), el strength de la senal recibe un bonus de hasta +15%. Esto mejora el win rate 3-7% sin reducir el numero de trades.

4.15 Kelly Criterion (Sizing Optimo)

Si hay 50+ trades de historial, el sizing usa Half-Kelly capped (0.5% a 3%) en vez del 2% fijo. Kelly se calcula por estrategia separadamente, adaptandose al edge real de MR.

4.16 Proteccion de Microestructura

El RiskManager bloquea entradas de MR cuando VPIN es toxico (≥ 0.6) o Hawkes spike ratio ≥ 3.0 . El razonamiento: flujo informado (VPIN alto) puede romper la reversion, causando perdidas cuando el precio no revierte.

5. Estrategia: Trend Following

Trend Following captura movimientos direccionales del mercado entrando en rompimientos y cruces de medias moviles. Usa trailing stops para dejar correr las ganancias y cortar perdidas rapido.

Regimen Optimo

TRENDING_UP, TRENDING_DOWN, BREAKOUT - Mercados con direccion clara y momentum fuerte.

5.5 Senales de Entrada

Se requiere al menos UNA de estas condiciones de entrada, mas filtros:

- Cruce de EMA: EMA rapida (12) cruza sobre EMA lenta (26) - senal principal
- Breakout: precio rompe Bollinger Band superior (long) o inferior (short)
- Filtro de regimen: debe coincidir con TRENDING_UP/DOWN o BREAKOUT
- Filtro de volumen: vol_ratio > 1.2 (volumen por encima de su media de 20 periodos)
- Filtro de momentum: momentum > 0 para long, < 0 para short (confirmacion direccional)

5.6 Trailing Stop

En vez de stop loss fijo, TF usa un trailing stop que se mueve a favor de la posicion:

```
trail_distance = ATR * tf_atr_mult_trail (default: ATR * 2.0)

LONG:  new_stop = max(current_stop, price - trail_distance)
SHORT: new_stop = min(current_stop, price + trail_distance)

Si price <= new_stop (long) o price >= new_stop (short) -> cierre
```

5.7 Take Profit

Ratio 3:1 respecto al riesgo del stop loss:

```
LONG:  take_profit = price + 3.0 * (price - stop_loss)
SHORT: take_profit = price - 3.0 * (stop_loss - price)
```

5.8 Ajuste por Fuerza de Tendencia

El capital asignado se reduce proporcionalmente al ADX (fuerza de tendencia). Con ADX bajo, la posicion es mas pequena; con ADX alto, es mas agresiva:

```
strength = min(ADX / 50, 1.0)
adjusted_capital = allocated_capital * strength
```

6. Estrategia: Market Making

Market Making captura el spread bid-ask colocando ordenes limite a ambos lados del precio. Usa el modelo Avellaneda-Stoikov mejorado para calcular spreads optimos que balancean rentabilidad contra riesgo de inventario y adverse selection.

Regimen Optimo

RANGING, UNKNOWN - Mercados tranquilos donde el spread se captura sin riesgo direccional excesivo.

6.5 Motor Avellaneda-Stoikov Mejorado

El motor calcula precios optimos de bid/ask con estas formulas:

```
# Precio de reserva (ajustado por inventario)
reservation_price = mid_price - inventory * ATR * gamma * T

# Spread optimo
ATR_bps = (ATR / mid_price) * 10,000
kappa_factor = 1 / (0.5 + effective_kappa * 0.333)
optimal_spread = ATR_bps * effective_gamma * kappa_factor * (0.5 + T)

# Inventory skew (suave, no lineal)
skew = tanh(inventory_ratio * 2) * ATR * 0.5

# Precios finales
bid = reservation_price - spread/2 - skew
ask = reservation_price + spread/2 - skew
```

6.6 Ajuste Dinamico de Gamma

Gamma (aversion al riesgo) se incrementa automaticamente con VPIN y Hawkes, lo que ensancha el spread como proteccion:

- VPIN multiplier: 1.0x (VPIN=0) hasta 3.0x (VPIN>=0.8)
- Hawkes multiplier: 1.0x (normal) hasta 2.5x (spike ratio 7x)
- Gamma combinado: puede llegar a ~7.5x el base, produciendo spreads de hasta 100 bps

6.7 Filtros de Seguridad

- PAUSA COMPLETA: Si VPIN >= 0.8 Y Hawkes spike simultaneo, no se generan quotes
- Reduccion de tamano: risk_score > 0.3 reduce sizing hasta 60%
- Spread minimo: siempre >= 2x fee medio (7 bps) para cubrir costos
- Spread maximo: 100 bps (defensivo, no captura spread pero protege capital)
- Inventory limits: no genera bids si inventory >= max, no genera asks si inventory <= -max

6.8 Unwind de Inventario

Cuando el regimen cambia y MM se desactiva (ej: RANGING -> TRENDING), el inventario existente se cierra con una market order. Sin esto, la posicion de MM quedaria expuesta a riesgo direccional sin limites.

7. Indicadores de Microestructura

BotStrike incorpora tres indicadores avanzados de microestructura de mercado que operan a nivel de tick individual (no solo por barra). Estos indicadores alimentan las decisiones de todas las estrategias y del risk manager.

7.5 VPIN (Volume-Synchronized Probability of Informed Trading)

VPIN mide la probabilidad de que traders informados esten operando. Un VPIN alto indica desequilibrio en el flujo de ordenes, lo que sugiere que el precio puede moverse bruscamente.

- Algoritmo: Bulk Volume Classification clasifica cada trade como compra/venta
- Agrupacion en buckets de volumen fijo (BTC: \$50k, ETH: \$10k, ADA: \$500)
- $VPIN = \text{media de } |\text{buy_vol} - \text{sell_vol}| / \text{total_vol} \text{ sobre } N \text{ buckets}$
- Threshold toxico: 0.6 (flujo informado detectado)
- Impacto: MR bloqueado, MM ensancha spread o pausa, TF puede confirmar momentum

7.6 Hawkes Process (Deteccion de Picos de Actividad)

El proceso de Hawkes modela la auto-excitacion en eventos de trading. Cuando un cluster de trades ocurre, la intensidad sube temporalmente, detectando picos anómalos de actividad.

```
lambda(t) = mu + SUM[ alpha * exp(-beta * (t - t_i)) ]
```

```
mu      = 1.0    (intensidad base)
alpha   = 0.5    (factor de excitacion)
beta    = 2.0    (tasa de decaimiento)
spike   = intensity > mu_original * 2.5
```

Nota: el threshold de spike usa mu ORIGINAL (fijo), no el adaptativo. Esto evita que el mu adaptativo absorba los spikes, haciendolos indetectables.

7.7 Kyle Lambda (Market Impact Estimation)

Kyle Lambda estima cuanto mueve \$1 de volumen signed el precio permanentemente. Es el eslabon entre deteccion de toxicidad (VPIN/Hawkes) y optimizacion de ejecucion. Lambda alto = mercado ilíquido o dominado por informed traders.

```
# Estimacion rolling incremental
lambda = Cov(delta_P, Q) / Var(Q)

delta_P = cambio de precio en bps
Q = signed volume en USD (buy=+, sell=-)

# Smoothing
lambda_ema = EMA(lambda, span=100)

# Impact estimation
permanent_impact = lambda_ema * sqrt(size/depth)
```

Integracion en el sistema:

- A-S Engine: gamma escala 1.0x a 1.5x con impact_stress (lambda alto = spreads mas anchos)
- Smart Router: penaliza market orders sumando permanent_impact al costo estimado
- Risk Manager: impact_stress ≥ 1.5 bloquea trades, > 0.5 reduce sizing
- Slippage Model: componente 8 -- permanent impact = $\lambda * \sqrt{\text{size/depth}}$
- Paper Simulator: aplica permanent impact en fills simulados
- Adverse Selection: registra fills y mide mark-to-market tras 5 minutos

Impact Stress Score

0.0 = normal, 0.5 = moderado (reduce sizing), 1.5+ = extremo (bloquea trades). Se calcula como $\lambda_{\text{ema}} / 2.0$, capped a 2.0.

7.8 Risk Score Combinado

Los indicadores se combinan en un risk_score unico (0 a 1):

```
vpin_score = min(vpin / 0.8, 1.0)
hawkes_score = min((spike_ratio - 1) / 3, 1.0)
risk_score = max(vpin_score, hawkes_score)
```


8. Gestion de Riesgo

El RiskManager es la ultima linea de defensa antes de la ejecucion. Toda senal generada por las estrategias DEBE pasar por validacion de riesgo (excepto unwinds de emergencia).

8.4 Capas de Proteccion

Capa	Parametro	Accion
Microestructura	VPIN toxico / Hawkes spike	Bloquea MR, reduce sizing 50%
Funding Rate	≥ 5 bps/8h contra posicion	Bloquea entrada
Funding Rate	≥ 1 bps/8h contra posicion	Reduce sizing 30%
Circuit Breaker	Drawdown $> 80\%$ del max (12%)	Pausa 5 minutos
Max Drawdown	$\geq 15\%$ del equity peak	Bloquea todas las senales
Exposicion Total	$> 80\%$ del equity	Bloquea senales nuevas
Exposicion por Activo	$> \text{max_position_usd}$	Reduce tamano
Leverage	Margen $> 50\%$ equity	Reduce tamano
Perdidas Consecutivas	≥ 3 consecutivas	Reduce sizing exponencial
Stop Loss Dinamico	Drawdown $> 50\%$ max	Aprieta SL hasta 30%
Risk of Ruin	RoR $> 3\%$	Reduce sizing 50%. RoR $> 10\%$: pausa total
Volatility Targeting	Vol realizada vs 15% target	Escala posiciones (0.5x a 2.0x)
Correlation Stress	Avg corr > 0.85	Reduce exposicion hasta 60%
Impact Stress (Kyle Lar	$\text{lambda_ema} > \text{threshold}$	Stress ≥ 1.5 : bloquea. > 0.5 : reduce sizing
Kelly Criterion	50+ trades historial	Half-Kelly capped (0.5% a 3%)

8.5 Position Sizing

El tamano de posicion se calcula por riesgo, no por capital disponible:

```

risk_amount = capital * risk_per_trade_pct  (2%)
risk_per_unit = |price - stop_loss|
size_units = risk_amount / risk_per_unit
max_units = (capital * leverage) / price
final_size = min(size_units, max_units)

```

8.6 Reduccion por Perdidas Consecutivas

Tras 3+ perdidas consecutivas, el tamano se reduce exponencialmente:

```

reduction = 0.5 ^ (consecutive_losses - 2)

```

```

3 perdidas: 50% del tamano normal
4 perdidas: 25% del tamano normal
5 perdidas: 12.5% del tamano normal

```

9. Gestion de Portfolio

El PortfolioManager distribuye el capital entre estrategias y simbolos de forma dinamica, considerando el regimen actual, el performance historico de cada estrategia, y el nivel de drawdown.

9.4 Asignacion por Regimen

Regimen	Mean Reversion	Trend Following	Market Making
RANGING	45%	10%	45%
TRENDING_UP	10%	70%	20%
TRENDING_DOWN	10%	70%	20%
BREAKOUT	5%	85%	10%
UNKNOWN	30%	40%	30%

9.5 Factores de Ajuste

- Performance factor (0.5 a 1.5): sigmoid basado en PnL promedio por trade de la estrategia
- Drawdown factor (0.3 a 1.0): reduce asignacion general conforme drawdown aumenta
- Risk Parity blend (30%): inverse-vol weighting ajusta pesos por volatilidad realizada
- Symbol share: distribucion equitativa entre simbolos (1/N)

```
# Base allocation
allocation = equity * regime_weight * perf_factor * dd_factor * symbol_share

# Risk Parity adjustment (70% regime + 30% inverse-vol)
rp_ratio = risk_parity_weight / neutral_weight
base_weight = 0.7 * regime_weight + 0.3 * regime_weight * min(rp_ratio, 2.0)
```

9.6 Desactivacion de Estrategias

Una estrategia se desactiva automaticamente si: su peso por regimen es < 8% (no vale la pena operar con tan poco capital), o su performance factor < 0.6 (esta perdiendo consistentemente).

10. Ejecucion de Ordenes y Paper Trading

10.5 Smart Order Router

La decision limit vs market NO es hardcodeada - usa un modelo de costos que compara el costo esperado de cada alternativa y elige la mas barata:

```
# Costo de MARKET order
market_cost = half_spread + size_impact + taker_fee + lambda_impact

# Costo de LIMIT order
limit_cost = (1-P(fill)) * opportunity_cost + P(fill) * (maker_fee - price_improvement)

# Decision
decision = MARKET if market_cost <= limit_cost else LIMIT
```

Casos especiales que se resuelven sin modelo:

Contexto	Decision	Razon
Market Making	LIMIT + post_only	Capturar maker rebate siempre
Salida urgente (SL/TP/exit)	MARKET	Prioridad = velocidad
Spread < 3 bps	MARKET	Spread tan tight que limit no mejora
Orden > \$10k	TWAP split	Reduce market impact (sqrt model)

10.6 Fill Probability Model

Estima P(fill) para ordenes limit usando 5 factores combinados con logistic function:

- Distancia al mid (normalizada por spread): mas lejos = menor prob
- Volatilidad (ATR): mayor vol = mas prob de que precio toque nuestro nivel
- Queue depth: mas capital delante = menor prob de fill
- Trade intensity: mas trades/sec = mas prob de que nos toque
- Horizonte temporal: mas tiempo = mas prob (sqrt scaling, Brownian motion)

10.7 Modelo de Slippage Avanzado (8 Componentes)

El slippage usa un modelo de 8 factores calibrables:

1. Spread component: half_spread (market) o 0 (limit)
2. Size impact: $\sqrt{\text{size/depth}} * 3 \text{ bps}$ (Almgren-Chriss concavo)
3. Volatility scaling: $0.5 + \text{ATR_bps}/20 * 0.5$
4. Hawkes impact: $(\text{ratio}-1) * 0.8 \text{ bps}$ si spike
5. Regime multiplier: RANGING=0.8x a BREAKOUT=2.0x
6. OBI adverse selection: imbalance_contra * 2 bps
7. VPIN toxicity premium: $(\text{vpin}-0.4) * 3 \text{ bps}$ si toxico
8. Permanent impact: $\lambda * \sqrt{\text{size/depth}}$ (Kyle Lambda)

10.8 Paper Trading Simulator

El paper simulator reemplaza al exchange en modo --paper. La clave de diseno es que los fills simulados fluyen por el MISMO pipeline que los fills reales:

- Aplica slippage dinamico (no fijo) basado en regimen y tamano
- Verifica SL/TP en CADA tick de precio via on_price_update()
- Produce objetos Trade identicos a los del exchange real
- Entry: fee=0, pnl=0. Exit: fee=ambos lados, pnl=gross-fees (identico al exchange)
- Tracked via trade_db con source='paper' para analytics separados

11. Sistema de Backtesting

11.4 Backtester Basico (Bar-by-Bar)

Itera sobre barras OHLCV, aplica indicadores, detecta regimen, genera senales, y simula fills con slippage y fees. Incluye verificacion de SL/TP intra-bar y liquidaciones por leverage.

11.5 Backtester Realista (Tick-by-Tick)

Replica EXACTAMENTE el loop de live trading. Diferencias clave vs el basico:

- Alimenta microestructura tick-by-tick ANTES de procesar cada barra
- Usa PortfolioManager real con asignacion dinamica por regimen
- Usa RiskManager completo con TODOS los filtros (drawdown, exposure, micro, funding)
- Distingue fees de maker (MM) vs taker (MR/TF)
- Produce JSONL identico al de produccion (signal, trade, micro, regime, portfolio)
- Soporta orderbook real del collector (no solo simulado con ATR)

11.6 Metricas de Resultado

Metrica	Formula	Uso
Net PnL	Suma de PnL de todos los trades	Rentabilidad absoluta
Win Rate	Trades ganadores / Total trades	Consistencia
Profit Factor	Gross profit / Gross loss	Eficiencia (>1 = rentable)
Sharpe Ratio	$\text{mean}(\text{daily_ret}) / \text{std}(\text{daily_ret}) * \text{sqrt}(252)$	Riesgo-ajustado
Calmar Ratio	Return / Max Drawdown	Retorno vs peor caida
Max Drawdown	Maxima caida desde peak de equity	Peor escenario historico

12. Integracion con Strike Finance

12.4 Autenticacion Ed25519

Strike Finance usa autenticacion criptografica Ed25519 (no HMAC-SHA como Binance). Cada request autenticado requiere:

```
message = f'{METHOD}:{PATH}:{TIMESTAMP}:{NONCE}:{SHA256(BODY)}'  
signature = Ed25519_sign(message, private_key)
```

Headers:

```
X-API-Wallet-Public-Key: <public_key_hex>  
X-API-Wallet-Signature: <signature_hex>  
X-API-Wallet-Timestamp: <unix_seconds>  
X-API-Wallet-Nonce: <uuid4>
```

12.5 WebSocket Dual

- Market WS (wss://api.strikefinance.org/ws/price): trades, depth, markprice publicos
- User WS (wss://api.strikefinance.org/ws/user-api): fills, posiciones, balance (autenticado)
- Reconexion automatica con exponential backoff (1s a 60s)
- Re-suscripcion de canales tras cada reconexion
- Manejo de frames con multiples JSON separados por newline

12.6 Rate Limiting

Token bucket rate limiter: 50 requests por 10 segundos. Si se alcanza el limite, el request espera automaticamente hasta que haya capacidad.

13. Pipeline de Datos

13.3 Recoleccion Continua

El data collector (python main.py --collect-data) recolecta datos continuamente de Strike Finance MAINNET, independientemente de si el bot opera en testnet:

Tipo de Dato	Fuente Primaria	Backup	Almacenamiento	Frecuencia Flush
Trades	WebSocket	REST /v2/trades cada 15s	Parquet diario por simbolo	30s
Klines 1m	WebSocket	REST /v2/klines cada 60s	Parquet incremental	60s
Orderbook	WebSocket depth	REST /v2/depth cada 10s	Parquet diario por simbolo	30s
Stats	REST periodico	-	JSON metadata	300s

13.4 Formato de Almacenamiento

- Parquet con compresion zstd (40-60% menos que snappy)
- Organizacion: data/{tipo}/{simbolo}/{fecha}.parquet
- Klines: data/klines/{simbolo}/1m.parquet (incremental)
- Deduplicacion por trade_id o timestamp+price+qty
- Catalogo JSON con metadata de cada dataset (filas, fechas, tamano)

14. Configuracion y Parametros

14.3 Parametros Globales (TradingConfig)

Parametro	Valor	Descripcion
initial_capital	\$100,000	Capital inicial para trading/backtest
max_drawdown_pct	15%	Drawdown maximo permitido (bloquea trading)
max_leverage	20x	Leverage maximo global
max_total_exposure_pct	80%	Exposicion maxima vs equity
risk_per_trade_pct	2%	Riesgo por trade (position sizing)
maker_fee	0.02%	Fee para ordenes maker (MM)
taker_fee	0.05%	Fee para ordenes taker (MR/TF)
slippage_bps	2 bps	Slippage base estimado
funding_rate_warn	1 bps/8h	Reduce sizing 30% contra funding
funding_rate_block	5 bps/8h	Bloquea entradas contra funding
data_stale_warn_sec	30s	Warning si datos > 30s sin actualizar
data_stale_block_sec	120s	No opera si datos > 2min sin actualizar
strategy_interval_sec	5s	Frecuencia del loop MR/TF
mm_interval_sec	0.5s	Frecuencia del loop Market Making

14.4 Parametros por Simbolo (SymbolConfig)

Parametro	BTC-USD	ETH-USD	ADA-USD
leverage	10x	15x	20x
max_position_usd	\$20,000	\$15,000	\$5,000
mr_zscore_entry	2.0	2.0	2.0
mr_lookback	100	100	100
mr_atr_mult_sl	1.5	1.5	1.5
mr_atr_mult_tp	2.5	2.5	2.5
tf_ema_fast / slow	12 / 26	12 / 26	12 / 26
tf_atr_mult_trail	2.0	2.0	2.0
mm_base_spread_bps	10	10	10
mm_order_levels	3	3	3
mm_order_size_usd	\$500	\$500	\$500
vpin_bucket_size	\$50,000	\$10,000	\$500
hawkes_mu / alpha / beta	1.0 / 0.5 / 2.0	1.0 / 0.5 / 2.0	1.0 / 0.5 / 2.0

15. Tick Quality Guards (WebSocket)

El sistema incorpora guards de calidad de ticks inspirados en mejores practicas de trading de alta frecuencia. Estos filtros protegen contra datos corruptos o stale que podrian causar fills incorrectos, hedges erroneos o entradas en precios falsos.

15.3 Guards Implementados

Guard	Parametro	Comportamiento
Warmup Period	5 segundos	Descarta TODOS los ticks en los primeros 5s post-conexion WS
First Tick Skip	1 tick/simbolo	Primer tick por simbolo post-conexion es snapshot cacheado, se descarta
Stale Tick Guard	5% max delta	Rechaza ticks con delta > 5% vs ultimo precio aceptado
Jitter EMA	alpha=0.1	Monitorea intervalo promedio entre ticks (diagnostico, no filtro)

15.4 Diseno

- Guards solo activos DESPUES de `on_ws_connected()` (cuando WS real se conecta)
- Backtesting, tests y REST init NO pasan por guards (`ws_connect_time=0`)
- Metricas de calidad disponibles via `get_tick_quality_stats()`
- Logging periodico en `_metrics_loop` cada 60 segundos
- `on_ws_connected()` se dispara automaticamente en cada reconexion del WS

Limitaciones de Datos

Cuando el collector se detiene (PC apagado, crash), los trades y orderbook de ese periodo se pierden. Solo las klines (velas 1m) se pueden recuperar via REST con `startTime/endTime`. El collector NO hace backfill automatico al arrancar.

16. Microprice y Order Book Alpha

16.3 Microprice (Stoikov 2018)

El mid-price clasico $(best_bid + best_ask) / 2$ ignora la informacion contenida en las cantidades de cada lado del book. El microprice corrige esto ponderando por cantidades inversas:

```
microprice = ask * (bid_qty / (bid_qty + ask_qty))
            + bid * (ask_qty / (bid_qty + ask_qty))
```

Intuicion:

```
bid_qty >> ask_qty -> mucho soporte -> precio justo sube hacia ask
ask_qty >> bid_qty -> presion venta -> precio justo baja hacia bid
bid_qty == ask_qty -> microprice == mid_price (caso clasico)
```

BotStrike implementa 3 niveles de sofisticacion:

- Level-1: Solo top-of-book (mas rapido, baseline institucional)
- Multi-Level: Pondera N niveles con decay exponencial (0.6^{nivel})
- Adjusted: Incorpora trade intensity buy/sell y OBI momentum delta

Impacto

Microprice reemplaza mid_price como fair value en: reservation price A-S, referencia de slippage, metadata de senales para smart routing.

16.4 Order Book Imbalance (OBI)

OBI mide la presion relativa de compra vs venta en los niveles del orderbook. Es un predictor estadisticamente significativo de movimiento a corto plazo en crypto.

```
# Simple imbalance
OBI = (bid_depth - ask_depth) / (bid_depth + ask_depth)

# Weighted (niveles cercanos pesan mas, decay=0.5)
weighted_OBI = sum(bid_usd * w) - sum(ask_usd * w) / total_weighted

# Delta (mas predictivo que nivel absoluto)
delta = current_OBI - previous_OBI
```

Integracion en estrategias:

Estrategia	Uso del OBI	Efecto
Mean Reversion	Confirmacion de reversal	Strength bonus hasta +15% si OBI favorable
Trend Following	Boost de confianza	Strength bonus hasta +15% si OBI en direccion
Market Making	Spread skew	Desplaza fair value segun presion del book

17. Modelos Cuantitativos Avanzados

17.7 Volatility Targeting

Escala la exposicion total del portfolio para mantener una volatilidad anualizada constante. Cuando la vol realizada sube, reduce posiciones. Cuando baja, aumenta. Usado por CTAs profesionales para estabilizar el Sharpe ratio.

```
scalar = target_vol / realized_vol(20d)
scalar = clamp(scalar, 0.5, 2.0)

target_vol = 15% anualizado (configurable)
Se aplica a TODAS las allocations antes del risk manager
```

17.8 Kelly Criterion (Half-Kelly Capped)

Sizing optimo de posiciones basado en el edge real del sistema. Se calcula por estrategia con ventana rolling de 200 trades.

```
f* = (p * b - q) / b
p = win_rate, q = 1-p, b = avg_win/avg_loss

risk_pct = clamp(f*/2, floor=0.5%, ceiling=3%)
Activacion: 50+ trades de historial. Default: 2% si insuficiente.
```

17.9 Risk of Ruin

Calcula la probabilidad de alcanzar el max drawdown. Se recalcula tras cada trade y auto-throttelea el sistema.

```
# Formula analitica
edge = win_rate * payoff_ratio - (1 - win_rate)
RoR = ((1-edge)/(1+edge))^capital_units

# Auto-control
RoR > 3% -> reduce sizing 50%
RoR > 10% -> pausa trading completamente
```

17.10 Monte Carlo Bootstrap

Simulacion de equity curves por resamplio de trades historicos. NO usa GBM - preserva la estructura de dependencia real.

- 10,000 simulaciones por default (configurable)
- Output: percentiles de equity final (p5/median/p95)
- Max drawdown distribution (median y p95)
- Probabilidad de ser rentable y probabilidad de ruin

17.11 Correlation Regime

Detecta cuando la correlacion entre activos sube por encima de 0.85 (stress mode). En crypto, las correlaciones saltan a ~1.0 durante crashes, eliminando la diversificacion.

```
avg_corr = mean(pairwise_correlations(BTC, ETH, ADA))
if avg_corr > 0.85:
    stress_factor = max(0.4, 1 - (corr-0.85)/(1-0.85) * 0.6)
    # Reduce exposicion total automaticamente
```

17.12 Risk Parity (Covarianza)

Inverse-volatility weighting que asigna mas capital a los buckets menos volatiles. Se combina 70/30 con los pesos de regimen para no perder la logica existente.

```
weight_i = (1/vol_i) / sum(1/vol_j)
final = 0.7 * regime_weight + 0.3 * regime_weight * rp_ratio
```

18. Execution Intelligence

Capa de ejecucion nivel institucional que reemplaza la logica hardcodeada de order type con modelos de costos, probabilidad de fill, y algoritmos de ejecucion.

18.7 Queue Position Model

Estima la posicion en la cola para ordenes limit. En un CLOB, las ordenes se ejecutan en orden price-time priority. Si hay \$50k delante, necesitas que se ejecuten antes.

```
queue_ahead = price_level_depth_usd
consume_rate = trade_rate * avg_trade_size * 0.5 # Solo tu lado
time_to_front = queue_ahead / consume_rate
```

18.8 Spread Predictor

Predice el spread futuro usando features del mercado:

- EMA del spread historico (baseline con mean reversion)
- Volatilidad (ATR alto -> spread ancho): factor 0.7x a 1.5x
- VPIN (flujo toxico -> spread sube): factor 1.0x a 1.4x
- Hawkes (actividad anomala -> spread cambia): factor hasta 1.3x
- OBI (imbalance alto -> spread se ensancha): factor hasta 1.3x

18.9 Trade Intensity Model (Bidireccional)

A diferencia del Hawkes unidireccional (que suma todos los trades), este modelo separa buy vs sell trades para entender la presion direccional del flujo. Alimenta el microprice ajustado y el fill probability model.

18.10 TWAP Engine

Para ordenes grandes (>\$10k), divide la ejecucion en N slices temporales para minimizar market impact. Impact es concavo (sqrt), asi que 4 ordenes de \$2.5k tienen menos impacto total que 1 de \$10k.

18.11 Execution Analytics

Post-trade analysis que mide calidad de ejecucion:

- Implementation shortfall: diferencia decision_price vs fill_price
- Fill rate: % de limit orders que se ejecutaron
- Slippage real vs modelado: calibra el slippage model empiricamente
- Latencia: tiempo entre envio de orden y fill
- Breakdown por estrategia y tipo de orden

18.12 Slippage Real Measurement

Cada Trade registra `expected_price` (de la senal), `fill_price` (real), y `actual_slippage_bps`. El `SlippageTracker` agrega estos datos y produce estadisticas por simbolo y regimen. Esto permite recalibrar el modelo de slippage del backtester con datos reales, previniendo que los backtests sean sistematicamente optimistas.

19. Testing y Calidad deCodigo

19.4 Suites de Tests

Suite	Tests	Cobertura
test_bug_fixes.py	52	Verificacion de cada bug fix aplicado (55+ fixes)
test_execution_intelligence.py	34	Microprice, FillProb, SmartRouter, TWAP, Analytics
test_core_functional.py	21	Indicadores, regimen, microestructura, types
test_strategies_functional.py	15	MR, TF, MM: senales, filtros, edge cases
test_self_audit.py	31	Integracion end-to-end, paper sim, backtester
test_functional.py	36	Pipeline completo, analytics, stress, walk-forward

Total: 153+ tests automatizados. 122 core pasando al 100%.

19.5 Auditorias deCodigo Realizadas

Se han realizado 11 sesiones de auditoria profunda del codigo, incluyendo una revision cuantitativa completa, un audit E2E, y un deep audit post-quant-upgrade que encontro y corrigio 10 bugs adicionales (2 critical, 2 high, 6 medium). Total: 80+ bugs corregidos. Cada fix tiene su test de verificacion correspondiente.

- CRITICAL: micro=None crash, testnet siempre True, equity double-count
- HIGH: MR doubling, Hawkes self-defeating, ADX bearish bias, paper PnL=0
- HIGH: Paper slippage sin regimen, MM sin risk checks, RSI NaN edge case
- MEDIUM: WS reconnect race, bar boundary tick leakage, ISO week year
- MEDIUM: Hawkes sin validacion estabilidad, maintenance margin 0.5% a 2%
- LOW: Imports no usados, Euler hardcodeado, profit_factor inf vs 9999.99

19.6 Revision Cuantitativa

Se verifico numericamente cada formula del sistema contra implementaciones de referencia:

- Indicadores (ATR, RSI, Z-score, ADX, Bollinger): 9/9 checks OK
- VPIN discriminacion: balanced=0.19 vs directional=1.0 (separacion correcta)
- Hawkes burst detection: 4.3x spike ratio detectado, decay verificado
- A-S spreads: monotonicamente crecientes $\text{calm}(10) < \text{vpin}(28) < \text{crisis}(50)$ bps
- A-S inventory skew: verificado numericamente (long=quotes bajan, short=suben)
- Position sizing, drawdown, liquidation: formulas verificadas con calculos manuales
- PnL arithmetic: $\text{sum}(\text{trades}) = \text{reported net_pnl}$ en backtests de 3000 barras
- Trade DB round-trip: datos preservados exactamente (0 perdida de precision)
- JSON serialization: todos los outputs del sistema son serializables